

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №5
«Интерполяция функции»
Вариант №2

Группа Р3208

Студент Горин Семён Дмитриевич

Преподаватель Рыбаков Степан Дмитриевич

Содержание

1. Цель работы	3
2. Порядок выполнения работы	3
2.1. Программная реализация задачи	3
2.1.1. Методы для реализации в программе	3
3. Рабочие формулы	4
3.1. Метод Рунге-Кутты 4-го порядка	4
3.2. Усовершенствованный метод Эйлера	4
3.3. Метод Милна	4
4. Листинг программы	4
5. Результаты выполнения программы	10
6. Выводы	11

1. Цель работы

решить задачу Коши для обыкновенных дифференциальных уравнений численными методами.

2. Порядок выполнения работы

2.1. Программная реализация задачи

- 1) В программе численные методы решения обыкновенных дифференциальных уравнений (ОДУ) должен быть реализован в виде отдельного класса / метода/функции;
- 2) Пользователь выбирает ОДУ вида $y' = f(x, y)$ (не менее трех уравнений), из тех, которые предлагает программа;
- 3) Предусмотреть ввод исходных данных с клавиатуры: начальные условия $y_0 = y(x_0)$, интервал дифференцирования $[x_0, x_n]$, шаг h , точность ε ;
- 4) Для исследования использовать одношаговые методы и многошаговые методы согласно варианту;
- 6) Составить таблицу приближенных значений интеграла дифференциального уравнения, удовлетворяющего начальным условиям, для всех методов, реализуемых в программе;
- 7) Для оценки точности одношаговых методов использовать правило Рунге;
- 8) Для оценки точности многошаговых методов использовать точное решение задачи: $\varepsilon = \max_{0 \leq i \leq n} |y_{i_T} - y_i|$;
- 9) Построить графики точного решения и полученного приближенного решения (разными цветами);
- 10) Программа должна быть протестирована при различных наборах данных, в том числе и некорректных.
- 11) Проанализировать результаты работы программы.

2.1.1. Методы для реализации в программе

- 1) Усовершенствованный метод Эйлера;
- 2) Метод Рунге-Кутты 4-го порядка;
- 3) Метод Милна;

3. Рабочие формулы

3.1. Метод Рунге-Кутта 4-го порядка

$$y_{i+1} = y_i + \left(\frac{1}{6}\right)(k_1 + 2k_2 + 2k_3 + k_4)$$

$$k_1 = h \cdot f(x_i, y_i)$$

$$k_2 = h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right)$$

$$k_3 = h \cdot f\left(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}\right)$$

$$k_4 = h \cdot f(x_i + h, y_i + k_3)$$

3.2. Усовершенствованный метод Эйлера

$$y_{i+1} = y_i + \left(\frac{h}{2}\right)(f(x_i, y_i) + f(x_{i+1}, y_i + h \cdot f(x_i, y_i))), i = 0, 1, \dots$$

3.3. Метод Милна

1) этап прогноза:

$$y_i^{\text{прогн}} = y_{i-4} + \left(4\frac{h}{3}\right)(2f_{i-3} - f_{i-2} + 2f_{i-1})$$

2) Этап коррекции:

$$y_i^{\text{корр}} = y_{i-2} + \left(\frac{h}{3}\right)(f_{i-2} + 4f_{i-1} + f_i^{\text{прогн}})$$

$$f_i^{\text{прогн}} = f(x_i, y_i^{\text{прогн}})$$

4. Листинг программы

```
1: package lab.math;
2:
3: import java.util.ArrayList;
4: import java.util.List;
5: import java.util.function.DoubleUnaryOperator;
6:
7: public class DifferentialEquationService {
8:
9:     public MethodResult solveWithEnhancedEuler(EquationDefinition
equation, double x0, double y0, double xn, double h, double epsilon) {
10:         validateGrid(x0, xn, h);
11:         double[] values = computeEnhancedEulerValues(equation, x0, y0,
xn, h);
12:
13:         List<ComputationPoint> points = buildPoints(equation, x0, y0,
h, values);
```

```

14:         double[] fineValues = computeEnhancedEulerValues(equation, x0,
y0, xn, h / 2.0);
15:         double rungeEstimate = 0.0;
16:         for (int i = 0; i < values.length; i++) {
17:             rungeEstimate = Math.max(rungeEstimate,
rungeEstimate(values[i], fineValues[i * 2], 2));
18:         }
19:         return new MethodResult("Enhanced Euler method", 2, h, points,
rungeEstimate, maxError(points));
20:     }
21:
22:     public MethodResult solveWithRungeKutta(EquationDefinition
equation, double x0, double y0, double xn, double h, double epsilon) {
23:         validateGrid(x0, xn, h);
24:         double[] values = computeRungeKutta4Values(equation, x0, y0,
xn, h);
25:
26:         List<ComputationPoint> points = buildPoints(equation, x0, y0,
h, values);
27:         double[] fineValues = computeRungeKutta4Values(equation, x0,
y0, xn, h / 2.0);
28:         double rungeEstimate = 0.0;
29:         for (int i = 0; i < values.length; i++) {
30:             rungeEstimate = Math.max(rungeEstimate,
rungeEstimate(values[i], fineValues[i * 2], 4));
31:         }
32:         return new MethodResult("Runge-Kutta 4 method", 4, h, points,
rungeEstimate, maxError(points));
33:     }
34:
35:     public MethodResult solveWithMilne(EquationDefinition equation,
double x0, double y0, double xn, double h, double epsilon) {
36:         validateGrid(x0, xn, h);
37:         int steps = (int) Math.round((xn - x0) / h);
38:         if (steps < 3) {
39:             throw new IllegalArgumentException("Milne method requires
at least three steps");
40:         }
41:
42:         double[] values = new double[steps + 1];
43:         values[0] = y0;
44:         bootstrapWithRungeKutta(equation, x0, h, values, Math.min(3,
steps));
45:
46:         for (int i = 3; i < steps; i++) {
47:             if (!isValidValue(values[i])) {
48:                 throw new IllegalArgumentException("Milne method
diverged at step " + i);
49:             }
50:             double xNext = x0 + (i + 1) * h;
51:             double fIm2 = equation.derivative().applyAsDouble(x0 + (i
- 2) * h, values[i - 2]);

```

```

52:         double fIm1 = equation.derivative().applyAsDouble(x0 + (i
- 1) * h, values[i - 1]);
53:         double fI = equation.derivative().applyAsDouble(x0 + i *
h, values[i]);
54:
55:         if (!isValidValue(fIm2) || !isValidValue(fIm1) || !
isValidValue(fI)) {
56:             throw new IllegalArgumentException("Milne method
encountered invalid derivative value at step " + i);
57:         }
58:
59:         double predictor = values[i - 3] + (4.0 * h / 3.0) * (2.0
* fIm2 - fIm1 + 2.0 * fI);
60:         double corrected = values[i - 1] + (h / 3.0) * (fIm1 + 4.0
* fI + equation.derivative().applyAsDouble(xNext, predictor));
61:
62:         if (!isValidValue(predictor) || !isValidValue(corrected))
{
63:             throw new IllegalArgumentException("Milne method
diverged at step " + i + " during prediction/correction");
64:         }
65:
66:         double previous;
67:         int iterations = 0;
68:         do {
69:             previous = corrected;
70:             corrected = values[i - 1] + (h / 3.0) * (fIm1 + 4.0 *
fI + equation.derivative().applyAsDouble(xNext, previous));
71:             iterations++;
72:         } while (Math.abs(corrected - previous) > epsilon &&
iterations < 20);
73:
74:         values[i + 1] = corrected;
75:     }
76:
77:     List<ComputationPoint> points = buildPoints(equation, x0, y0,
h, values);
78:     return new MethodResult("Milne method", 4, h, points, null,
maxError(points));
79: }
80:
81: // Enhanced Euler method (Runge-Kutta 2nd order)
82: private double[] computeEnhancedEulerValues(EquationDefinition
equation, double x0, double y0, double xn, double h) {
83:     int steps = (int) Math.round((xn - x0) / h);
84:     double[] values = new double[steps + 1];
85:     values[0] = y0;
86:     for (int i = 0; i < steps; i++) {
87:         double x = x0 + i * h;
88:         double y = values[i];
89:         if (!isValidValue(y)) {

```

```

90:         throw new IllegalArgumentException("Enhanced Euler
method diverged at step " + i + ", x=" + x + ": y=" + y);
91:     }
92:     double k1 = h * equation.derivative().applyAsDouble(x, y);
93:     double k2 = h * equation.derivative().applyAsDouble(x + h,
y + k1);
94:     if (!isValidValue(k1) || !isValidValue(k2)) {
95:         throw new IllegalArgumentException("Enhanced Euler
method encountered invalid derivative value at step " + i);
96:     }
97:     values[i + 1] = y + 0.5 * (k1 + k2);
98: }
99: return values;
100: }
101:
102: // Runge-Kutta 4th order
103: private double[] computeRungeKutta4Values(EquationDefinition
equation, double x0, double y0, double xn, double h) {
104:     int steps = (int) Math.round((xn - x0) / h);
105:     double[] values = new double[steps + 1];
106:     values[0] = y0;
107:     for (int i = 0; i < steps; i++) {
108:         double x = x0 + i * h;
109:         double y = values[i];
110:         if (!isValidValue(y)) {
111:             throw new IllegalArgumentException("Runge-Kutta method
diverged at step " + i + ", x=" + x + ": y=" + y);
112:         }
113:         double k1 = h * equation.derivative().applyAsDouble(x, y);
114:         double k2 = h * equation.derivative().applyAsDouble(x +
h / 2.0, y + k1 / 2.0);
115:         double k3 = h * equation.derivative().applyAsDouble(x +
h / 2.0, y + k2 / 2.0);
116:         double k4 = h * equation.derivative().applyAsDouble(x + h,
y + k3);
117:         if (!isValidValue(k1) || !isValidValue(k2) || !
isValidValue(k3) || !isValidValue(k4)) {
118:             throw new IllegalArgumentException("Runge-Kutta method
encountered invalid derivative value at step " + i);
119:         }
120:         values[i + 1] = y + (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0;
121:     }
122:     return values;
123: }
124:
125: // Bootstrap Runge-Kutta for Milne method
126: private void bootstrapWithRungeKutta(EquationDefinition equation,
double x0, double h, double[] values, int stepsToBootstrap) {
127:     double currentX = x0;
128:     for (int i = 0; i < stepsToBootstrap; i++) {
129:         double y = values[i];
130:         if (!isValidValue(y)) {

```

```

131:         throw new IllegalArgumentException("Bootstrap failed
at step " + i + ": y=" + y);
132:     }
133:     double k1 = h *
equation.derivative().applyAsDouble(currentX, y);
134:     double k2 = h *
equation.derivative().applyAsDouble(currentX + h / 2.0, y + k1 / 2.0);
135:     double k3 = h *
equation.derivative().applyAsDouble(currentX + h / 2.0, y + k2 / 2.0);
136:     double k4 = h *
equation.derivative().applyAsDouble(currentX + h, y + k3);
137:     if (!isValidValue(k1) || !isValidValue(k2) || !
isValidValue(k3) || !isValidValue(k4)) {
138:         throw new IllegalArgumentException("Bootstrap
encountered invalid derivative value at step " + i);
139:     }
140:     values[i + 1] = y + (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0;
141:     currentX += h;
142: }
143: }
144:
145: private List<ComputationPoint> buildPoints(EquationDefinition
equation,
146:                                     double x0,
147:                                     double y0,
148:                                     double h,
149:                                     double[] values) {
150:     DoubleUnaryOperator exact = equation.exactSolution(x0, y0);
151:     List<ComputationPoint> points = new
ArrayList<>(values.length);
152:     for (int i = 0; i < values.length; i++) {
153:         double x = x0 + i * h;
154:         double y = values[i];
155:         double exactValue = exact.applyAsDouble(x);
156:         points.add(new ComputationPoint(x, y, exactValue,
Math.abs(exactValue - y)));
157:     }
158:     return points;
159: }
160:
161: private double maxError(List<ComputationPoint> points) {
162:     double max = 0.0;
163:     for (ComputationPoint point : points) {
164:         max = Math.max(max, point.absoluteError());
165:     }
166:     return max;
167: }
168:
169: private void validateGrid(double x0, double xn, double h) {
170:     double steps = (xn - x0) / h;
171:     if (Math.abs(Math rint(steps) - steps) > 1e-9) {

```



```
172:         throw new IllegalArgumentException("The interval length
must be divisible by the step size");
173:     }
174: }
175:
176:     private double rungeEstimate(double coarseValue, double fineValue,
int order) {
177:         return Math.abs(fineValue - coarseValue) / (Math.pow(2.0,
order) - 1.0);
178:     }
179:
180:     private boolean isValidValue(double value) {
181:         return !Double.isNaN(value) && !Double.isInfinite(value);
182:     }
183: }
184:
```

5. Результаты выполнения программы

```

1: Choose an equation:
2:   1)  $y' = y + (1 + x) y^2$ 
3:   2)  $y' = x + y$ 
4:   3)  $y' = y - x^2 + 1$ 
5: Your choice: 1
6: Enter x0 [default 1.000000]:
7: Enter y0 [default -1.000000]:
8: Enter xn [default 1.500000]:
9: Enter step h [default 0.100000]:
10: Enter epsilon [default 0.000001]:
11:
12: Equation:  $y' = y + (1 + x) y^2$ 
13: Initial conditions:  $y(1.000000) = -1.000000$ 
14: Interval: [1.000000, 1.500000], step = 0.100000, epsilon = 1.000000e-06
15: Method: Enhanced Euler method
16: Step size: 0.100000
17: Runge estimate: 3.8572012099e-04
18: Max absolute error: 1.5052552099e-03
19:
20:      x          y(x) approx      y(x) exact      abs error
21: 1.000000    -1.00000000000    -1.00000000000    0.00000000000
22: 1.100000    -0.90995000000    -0.90909090909    0.00085909090
23: 1.200000    -0.8346160376     -0.83333333333    0.0012827043
24: 1.300000    -0.7706932440     -0.7692307692     0.0014624748
25: 1.400000    -0.7157909695     -0.7142857143     0.0015052552
26: 1.500000    -0.6681387752     -0.66666666667    0.0014721086
27: Method: Runge-Kutta 4 method
28: Step size: 0.100000
29: Runge estimate: 2.3392989351e-07
30: Max absolute error: 3.7232317983e-06
31:
32:      x          y(x) approx      y(x) exact      abs error
33: 1.000000    -1.00000000000    -1.00000000000    0.00000000000
34: 1.100000    -0.9090933148     -0.90909090909    0.0000024057
35: 1.200000    -0.8333367499     -0.83333333333    0.0000034166
36: 1.300000    -0.7692344925     -0.7692307692     0.0000037232
37: 1.400000    -0.7142893912     -0.7142857143     0.0000036769
38: 1.500000    -0.6666701275     -0.66666666667    0.0000034609
39: Method: Milne method
40: Step size: 0.100000
41: Max absolute error: 3.7232317983e-06
42:
43:      x          y(x) approx      y(x) exact      abs error
44: 1.000000    -1.00000000000    -1.00000000000    0.00000000000
45: 1.100000    -0.9090933148     -0.90909090909    0.0000024057
46: 1.200000    -0.8333367499     -0.83333333333    0.0000034166
47: 1.300000    -0.7692344925     -0.7692307692     0.0000037232
48: 1.400000    -0.7142847743     -0.7142857143     0.0000009400
49: 1.500000    -0.6666684137     -0.66666666667    0.0000017470
50: Charts were saved to build/charts/build/charts/y_y_1_x_y_2.png

```

Листинг 1 — Вывод программы в консоль

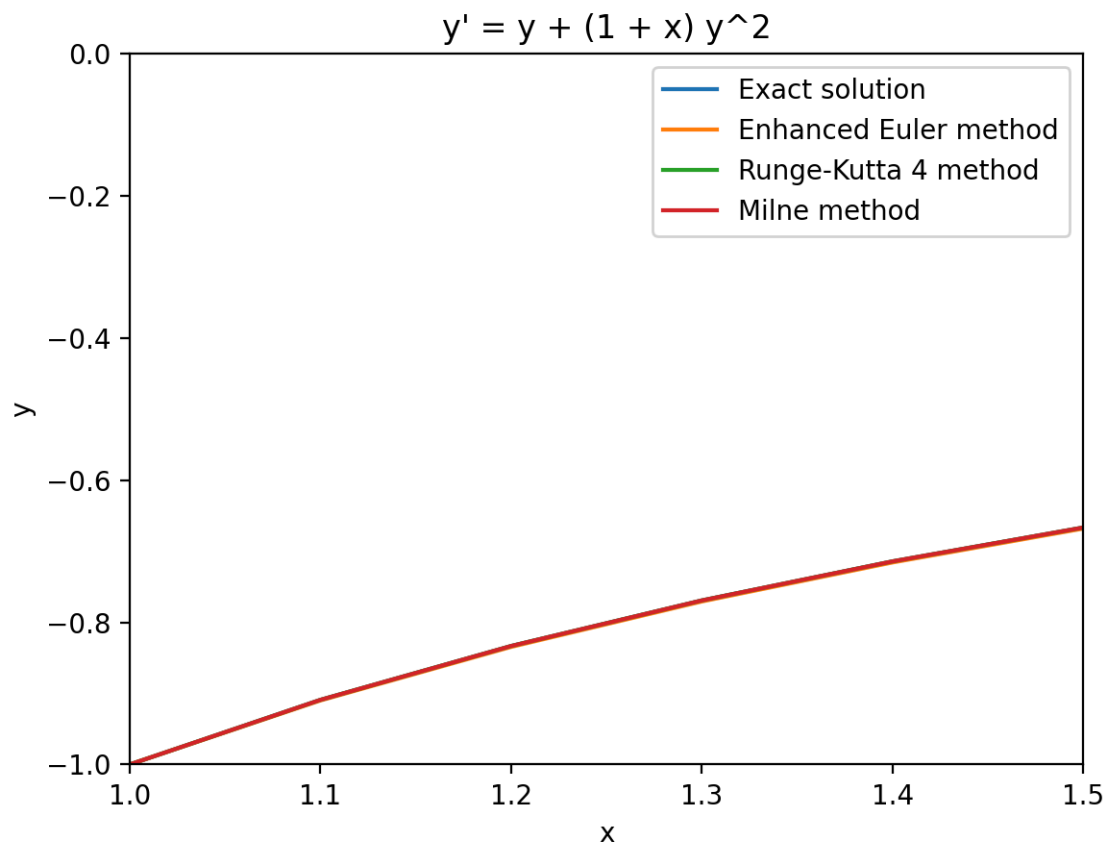


Рис. 1 — Полученный в результате работы график

6. Выводы

По итогу выполнения лабораторной работы были реализованы алгоритмы решения ОДУ: усовершенствованный метод Эйлера, метод Рунге-Кутты 4-го порядка и метод Милна. Проведено сравнение полученных приближенных решений с точным решением задачи Коши. На графике представлены точное решение и приближенные решения, полученные различными методами. Результаты показывают, что метод Рунге-Кутты 4-го порядка обеспечивает наибольшую точность среди рассмотренных методов.